Experiment 'p_4_1_c_powers_of_two_array' Results

June 3, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_1_c_powers_of_two_array
Natural language statement: Write a method with loops that computes all powers of 2 from 2^0 up to 2^20.
Method signature: p_4_1_c_powers_of_two_array() returns (powers: array<int>)

Ensures

- `powers.Length == 21`
- `forall i :: 0 <= i < powers.Length ==> powers[i] == pow(2, i)`

Functional Code Given

```
function pow(base: int, exp: int): int
  requires 0 <= exp
  decreases exp
{
  if exp == 0 then 1 else base * pow(base, exp - 1)
}
```

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method with loops that computes all powers of 2 from 2^0 up to 2^20.

The signature should be:

p_4_1_c_powers_of_two_array() returns (powers: array<int>)

The method should respect the following contract:

ensures powers.Length == 21, ensures forall i :: 0 <= i < powers.Length ==> powers[i] ==
    ↪ pow(2, i)

The contract uses the following dafny code:

function pow(base: int, exp: int): int
  requires 0 <= exp
  decreases exp
{
  if exp == 0 then 1 else base * pow(base, exp - 1)
}

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-Dme23bS6M65lsjFFxLdsduo5zV8eC
Model: gpt-5.5-2026-04-23
Created at: 1780485999
Finish reason: stop
Usage: 464 tokens in, and 3987 tokens out

```
{
  powers := new int[21];
  var i := 0;
  var p := 1;

  while i < 21
    invariant powers.Length == 21
    invariant 0 <= i <= 21
    invariant p == pow(2, i)
```

```
    invariant forall j :: 0 <= j < i ==> powers[j] == pow(2, j)
    decreases 21 - i
  {
    var oldI := i;
    var oldP := p;
    assert oldP == pow(2, oldI);

    powers[i] := p;
    assert powers[oldI] == pow(2, oldI);

    p := 2 * p;
    i := i + 1;

    assert i == oldI + 1;
    assert i > 0;
    assert i - 1 == oldI;
    assert pow(2, i) == 2 * pow(2, i - 1);
    assert p == 2 * oldP;
    assert p == pow(2, i);

    forall j | 0 <= j < i
      ensures powers[j] == pow(2, j)
    {
      if j == oldI {
        assert powers[j] == pow(2, j);
      } else {
        assert j < oldI + 1;
        assert j != oldI;
        assert j < oldI;
        assert powers[j] == pow(2, j);
      }
    }
  }
}
```

Final Program

```
function pow(base: int, exp: int): int
  requires 0 <= exp
  decreases exp
{
  if exp == 0 then 1 else base * pow(base, exp - 1)
}

method p_4_1_c_powers_of_two_array() returns (powers: array<int>)
       ensures powers.Length == 21
       ensures forall i :: 0 <= i < powers.Length ==> powers[i] == pow(2, i)
{
  powers := new int[21];
  var i := 0;
```

```
  var p := 1;

  while i < 21
    invariant powers.Length == 21
    invariant 0 <= i <= 21
    invariant p == pow(2, i)
    invariant forall j :: 0 <= j < i ==> powers[j] == pow(2, j)
    decreases 21 - i
  {
    var oldI := i;
    var oldP := p;
    assert oldP == pow(2, oldI);

    powers[i] := p;
    assert powers[oldI] == pow(2, oldI);

    p := 2 * p;
    i := i + 1;

    assert i == oldI + 1;
    assert i > 0;
    assert i - 1 == oldI;
    assert pow(2, i) == 2 * pow(2, i - 1);
    assert p == 2 * oldP;
    assert p == pow(2, i);

    forall j | 0 <= j < i
      ensures powers[j] == pow(2, j)
    {
      if j == oldI {
        assert powers[j] == pow(2, j);
      } else {
        assert j < oldI + 1;
        assert j != oldI;
        assert j < oldI;
        assert powers[j] == pow(2, j);
      }
    }
  }
}
```

Total Token Usage

Input tokens: 464
Output tokens: 3987
Reasoning tokens: 3582
Sum of 'total tokens': 4451

Experiment Timings

Overall Experiment started at 1780485999159, ended at 1780486077699, lasting 78540ms (78.54 seconds)
Iteration #1 started at 1780485999160, ended at 1780486077699, lasting 78539ms (78.54 seconds)